Text processing

Chunking - Various Datatypes

Processing -> clean, Transform,
↳ Easier algorithm

-> Extra
-> HTML , Encoding probl
-> Special , Boiler plate
-> dupic

1) Lowercasing -> Whitespace Normalization

-> Remove HTML
<h1> (_) </h1>

-> Remove Unwanted characters
!!! $$ #$ @@

- Tokenization -> Subword Tokenization

Reuse { [Unbelievable]
un believe able

Ugly -> Studying (studi) -> Stop words -> kill stop -> context
-> Stemming -> common root
-> lemmatization -> Dictionary
↳ Studies
↳ Study
↳

TF-IDF | ML

Modern → Transformers

Raw Text → Minimal cleaning → Model Tokenizer

Embedding Model ← Tokens

Chunking →

700 Page Document

One Giant piece

Small pieces

chunks

Document
- Chunk1
- chunk2
- chunk3
- ...
6

700 | Azure

Why ?

1) Manageability

↳ {100000 | words → process | unit}

↳ LLMs → context window

↳ Useless | 1 Page | few

What Azure file Storage

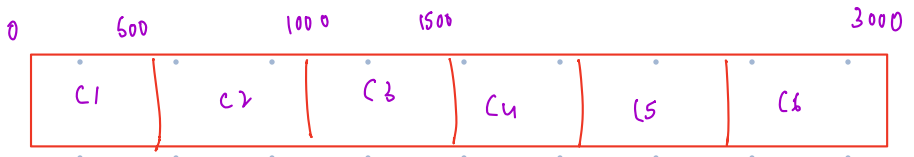
↳ 1 Page - 1 sentence

Representation

↳ 1000 Page - Small - chunks - One Topic

1) fixed Size chunking

Chunksize = 500 Tokens

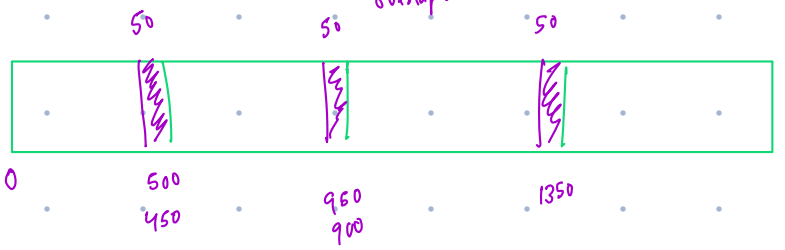


length = 500 / chunk

→ Sentence – Semantic / Two Meaning Split

2) fixed Size with Overlap

fs = 500
overlap = 50



→ Buffer Context (sharing

3) Sentence Base chunking

Split -around Sentence

Sentence - c1

Se2 - c2

Sen3 - C3

4) Paragraph Based Chunking

P1 , P2 , P3

↳ Natural Boundaries

→ Documentation
→ Articles
→ Manuals
→ Policies

5) Recursive chunking - 90%.

Document → Paragraph → Sentence ↓ character ← words

Algorithm → split → large meaningful Bound

eg :-

Try Paragraph Boundary
↓
Too large?
↓
Sentence Boundary
↓
Words

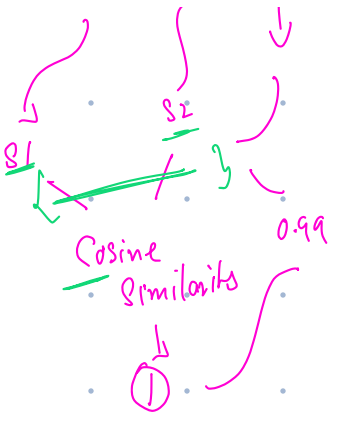
6) Semantic Chunking

Sentences - (S1) (S2) (S3) (S4) (S5)

Combine - chunk

Semantic - drifting

↳ New chunk



Sentence - chunks

↳ Lot of time - latency

→ cost

500 - Chunksize

↳ Very Small

↳ Precise Retrieval

focused meaning

less irrelevant information

Disa

→ Context

→ More chunks

→ Meta data overhead

fragmented

Very large

↳ more context

→ fewer Chunks

Disa →

less precise

→ irrelevant info

Chunksize - [200 - 1000]

Keyword Search

↳ TF-IDF → BM25

↳ lexical retrievals

↳ actual words

↳ Text

Keyword Search

1) Python - program ✓

2) Pyth com main ✓

3) Java is pro lang ✓

4) Azure provid cloud ✓

User Search

→ Python programming

↳ Python programming

D1 - ✓ ✓

D2 - ✓ x

D3 - x ✓

D4 - x x

Candidates → D1 D2 D3 → relevant

1 1 1

2 1 1

↳ Scoring

↳ TF-IDF come in

D1 → Py is pro lan

D2 → Py is pop pr lan

D3 → Py is use for ML

→ Python

D1 Py - Pgr large use ML ✓

D2 Py is used web deve ✓

D3 Py is used for DS ✓

→ Py →

ML → DI

Rare words → Generally - useful

↳ Identify relevent doc

↳ common

IDF

TF → Term frequency

↳ How frequently - Term occur

Document

→ Py Py Py Java

→ Python →

$$TF = \frac{\text{Number of term 't' appears}}{\text{total}}$$

$$= \frac{3}{4} = (0.75)$$

Python → DI

D1 → Python is python and pyth is programming

D2 → Python is useful

Q — Python — DI → Stronger sig

I

↳ There's

→ 'The' → occured DF

↳ IDF

↳ Inverse Document frequency

Intuition →

Word appearing - many documents

↓

less useful for distinguishing docum

word appearing - few document

↓

More useful for distinguish

Total documents = 1000

('The') → Term

↳ 950 Documents

('Doker')

↳ 30 Documents → Rare →

IDF →

$$IDF = \log\left(\frac{N}{DF(t)}\right)$$

= N - total Num Documents

= DF(t) → No of Documents Term(t)

N = 1000

Df(Doker) = 100

$$IDF(Doker) = \log\left(\frac{1000}{100}\right)$$

$$= \log(10)$$

Approx = 2.30

Now Suppose

DF(the) = 900

$$= \log\left(\frac{1000}{900}\right)$$

$$= 0.041$$

DF(Doker) = High IDF

DF(the) = low IDF

TF + IDF = TF-IDF

= TF (f,d) x IDF (f)

= if appearing freq - in this docu

and

if doesn't appear many other Documents

= How in Documents - TF

= How is distinguish from Rare - IDF

```
D1: Python is a programming language
D2: Python is used for machine learning
D3: Java is a programming language
```

N=3

Term- python

D1, D2, ~~D3~~

DF(python) =2

IDF(python) = log(3/2)

= 0.176

Machine = log(3/1)

= 0.477

= Python - 0.176

machine → 0.472 → discriminitive

Query - (Python machine)

(Numerical scores)

- D2 → Python Machine - ✓
- D1 → Python - ✓
- D3 - None - ⊗

High TF + high IDF → Important

High TF + low IDF → Common word

Low TF + High IDF → Useful

Low TF + low IDF → Not useful

TF-IDF problems

(1) Document length

D1 - 100 words

D2 - 10,000 words

→ Term (t) → 10 times

Same thing

(2) Repitalism

1 time → 1x

10 time → 10x

100 Time → 100x

word → Document - Relevant

(3) No Semantic Understanding

Car ≠ Automobile

TF-IDF → ≠

(4) Exact Vocab

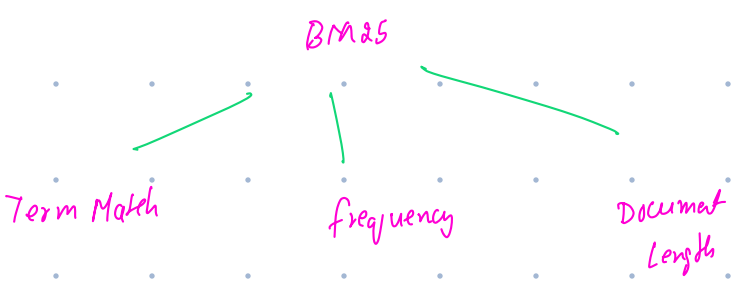
ML → Machine learning

BM25 → Traditional Algorithm

↳ Text Search

↳ Keyword Based

Core -Idea



BM25

$$= \sum IDF(t) \cdot \frac{TF(t,D)\,(k_1+1)}{TF(t,D) + k_1(1-b+b)}$$

Comonly

k_1 → 1.2 – 2.0

b → 0.75

IDF

K1 → Term frequency Saturati

Python

↳ DA → One time

DB → 10 times

DC → 100 times

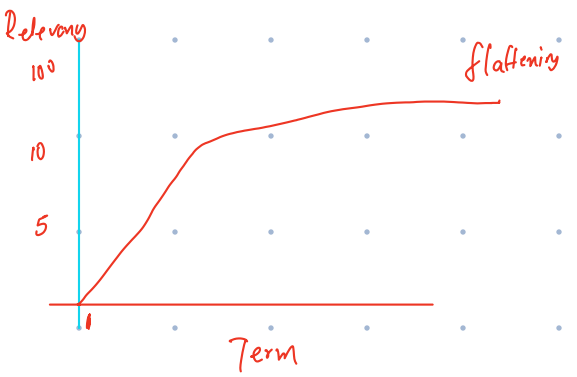
10 -DB - Evidence > 1 occurence

100 time — DC

↳ 100 time Relevant

Term frequency

K1 — Saturation



2) b → Document length Normalization

D1 — 100 words → Very important

D2 — 10,000 words → Uncommon

↳ Python - 10 times

BM25 - Account

↳ b → Normalization

	TF-IDF	BM25
Term Freq	✓	✓
Raw Term	✓	✓
Doc Freq	✓	✓
Semant	✗	✗
Docume length	✗	✓
Tf Saturation	✗	✗

Docim - 1

→ Chunking

↳ C1 C2 C3 C4 C5

D1 D2 D3 D4 D5

fixed size - 500 Token

Query → "What is Python, is programing?" => Keyword Math - (X)

↳ BM25

↳ Tokens

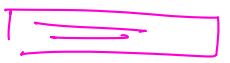
What - D1, D2

is - D1, D2,

↳ Python - D1, D2, D3, D4

↳ Program - D1, D3, D4

C1, C2, C3

↳ 

BM25 → K=3

↳ Retrive